ScoreMatch: Classical Composer Identification from MIDI

Nicholas Alforque

Thomas Geraci

Cameron Aljilani

Group 4

Shiley-Marcos School of Engineering, University of San Diego

AAI-511: Neural Networks and Deep Learning

Mirsardar Esnaeilli, PhD

August 10, 2026

Abstract

This project classifies classical music MIDI files from the Kaggle midi-classic-music dataset as works by Bach, Beethoven, Chopin, or Mozart. We compared classical machine-learning baselines built from 42 engineered symbolic features with two PyTorch deep-learning models that combined 20-FPS piano-roll inputs with the same engineered features: a convolutional neural network (CNN) and a convolutional neural network with long short-term memory (CNN-LSTM). After unreadable files were removed, the collection contained 1,628 valid MIDI files. Primary experiments used a balanced subset of 136 files per composer and a shared split of 392 training, 44 validation, and 108 test files. On the shared balanced test set, Random Forest achieved the strongest performance with 0.8611 accuracy and 0.8590 macro-F1. The 10-second CNN-LSTM reached 0.7685 accuracy and 0.7615 macro-F1, while the 10-second CNN reached 0.7315 accuracy and 0.7273 macro-F1. Increasing the CNN segment length to 15 seconds improved it to 0.7778 accuracy and 0.7776 macro-F1, making it the strongest pure deep-learning configuration. Stronger augmentation and hybrid CNN-engineered features did not surpass the best classical models. A full-dataset Random Forest increased accuracy to 0.8988 but produced a slightly lower macro-F1 of 0.8506. Overall, engineered symbolic MIDI features remained the most effective representation for this dataset.

Keywords: composer classification, MIDI, deep learning, CNN-LSTM, Random Forest

ScoreMatch: Classical Composer Identification from MIDI

Telling the difference between Bach and Mozart is something many trained listeners can do with familiar pieces, but identifying a composer automatically from symbolic music data is less straightforward (Raffel, 2016). Bach, Beethoven, Chopin, and Mozart share substantial harmonic and formal conventions, yet each composer also leaves stylistic patterns that can be represented computationally. Deep-learning models can learn useful representations directly from structured inputs, although their advantages are often strongest when sufficient training data are available (Goodfellow et al., 2016).

We approached the problem from two directions. First, we extracted numerical summary features from every MIDI file and trained classical machine-learning classifiers. Second, we converted the same files into fixed-length piano-roll matrices and trained convolutional and convolutional-recurrent networks. The balanced experiments used the same file-level train, validation, and test assignments for every model so performance differences could be attributed to model and representation choices rather than to different test samples.

Nicholas Alforque handled data collection, MIDI parsing, and feature extraction. Thomas Geraci developed and trained the CNN and CNN-LSTM models and prepared the shared deep-learning pipeline. Cameron Aljilani performed evaluation and follow-up experiments, including segment-length comparisons, hybrid features, and the full-dataset trial.

Method

Data Description

The source material was the public midi-classic-music collection on Kaggle (Fedorak, 2019). After unreadable files were removed, 1,628 valid MIDI files remained: 1,024 Bach, 212 Beethoven, 136 Chopin, and 256 Mozart. Because the source collection was strongly imbalanced

toward Bach, the primary experiments used a balanced subset of 136 files per composer, or 544 files total.

The balanced subset used a shared file-level split. Each composer contributed 98 training files, 11 validation files, and 27 test files, producing 392 training, 44 validation, and 108 testing files overall. The validation set was reserved for model selection and early stopping, while the test set remained untouched until final evaluation.

Feature Extraction and Preprocessing

The Kaggle data were already stored as MIDI files, so no score-to-MIDI conversion was required. Using pretty_midi (Raffel & Ellis, n.d.), we parsed each valid file and computed 42 numerical symbolic features. These included tempo, total duration, track counts, note density, pitch statistics, velocity statistics, note-duration statistics, interval statistics, polyphony rate, time-signature information, key information, and a 12-bin pitch-class distribution.

For the classical models, missing feature values were replaced with training-set medians. Logistic regression and the support vector machine also standardized the engineered features with statistics learned from the training set; the Random Forest did not require feature scaling. For the deep models, the imputer and standard scaler were fit only on the 392 training files and then applied to the validation and test sets.

A parallel piano-roll representation was created for the deep models. Each MIDI file was converted into a 128-pitch matrix at 20 frames per second (FPS), following the general use of structured piano representations in symbolic music modeling (Hawthorne et al., 2019). Deterministic fixed-length excerpts ensured that repeated runs used the same musical region from each file. Primary experiments used 10-second excerpts, and a later optimization experiment extended the inputs to 15 seconds.

Light augmentation was applied only to training piano rolls. It included small pitch shifts, short temporal shifts, brief temporal masking, slight velocity scaling, and low-level noise. A separate stronger augmentation experiment was later evaluated to determine whether additional variation improved generalization.

Models

Classical Baselines

Three scikit-learn classifiers were trained on the 42 engineered features (Pedregosa et al., 2011): logistic regression with balanced class weights, an RBF-kernel support vector machine with balanced class weights, and a Random Forest with 500 trees, balanced class weights, no maximum depth, and a minimum split size of two.

CNN

The CNN accepted a single-channel piano-roll input. Three convolutional blocks used 16, 32, and 64 filters with 3 × 3 kernels, batch normalization, ReLU activations, and max pooling, followed by adaptive average pooling. A separate fully connected branch processed the 42 engineered features. The two representations were concatenated before the final classifier. The completed CNN contained 317,348 trainable parameters (LeCun et al., 1998).

CNN-LSTM

The CNN-LSTM retained the convolutional front end and passed the temporal sequence to a bidirectional LSTM with hidden size 64. The recurrent representation was then fused with the engineered-feature branch before classification. The completed CNN-LSTM contained 129,316 trainable parameters (Hochreiter & Schmidhuber, 1997).

Training Procedure

The deep models were implemented in PyTorch and trained on a Google Colab GPU (Paszke et al., 2019). Seed 42 was set for Python, NumPy, and PyTorch to improve reproducibility. Primary CNN and CNN-LSTM runs used class-weighted cross-entropy, AdamW with a learning rate of 0.001 and weight decay of 0.0001, batch size 16, gradient clipping with a maximum norm of 5.0, and a learning-rate scheduler. Training was capped at 15 epochs, and early stopping with patience of three restored the weights from the best validation macro-F1.

Model evaluation included accuracy, macro precision, macro recall, macro-F1, per-composer classification reports, and confusion matrices. Macro-averaged measures were emphasized because they give each composer equal weight and are therefore especially useful when comparing the balanced experiment with the later imbalanced full-dataset trial.

Experiments and Results

Primary Shared-Test Comparison

Table 1 summarizes the primary balanced test-set comparison. The classical models and the 10-second deep models were evaluated on the same 108 held-out files. Random Forest produced the strongest overall performance, followed by the support vector machine. Among the 10-second deep models, the CNN-LSTM outperformed the CNN.

Table 1
Primary Performance on the Shared Balanced Test Set

Model	Accuracy	Macro Precision	Macro Recall	Macro-F1
Random Forest	0.8611	0.8670	0.8611	0.8590
Support Vector Machine	0.8056	0.8134	0.8056	0.8020
CNN-LSTM (10 s)	0.7685	0.7726	0.7685	0.7615
CNN (10 s)	0.7315	0.7500	0.7315	0.7273
Logistic Regression	0.7222	0.7212	0.7222	0.7192

The balanced Random Forest reached 0.8611 accuracy and 0.8590 macro-F1. Its advantage over the deep models indicates that the engineered MIDI summaries captured strong

composer-specific information even with a relatively small balanced training set. The support vector machine also remained competitive at 0.8020 macro-F1.

Figure 1

Balanced Random Forest Confusion Matrices

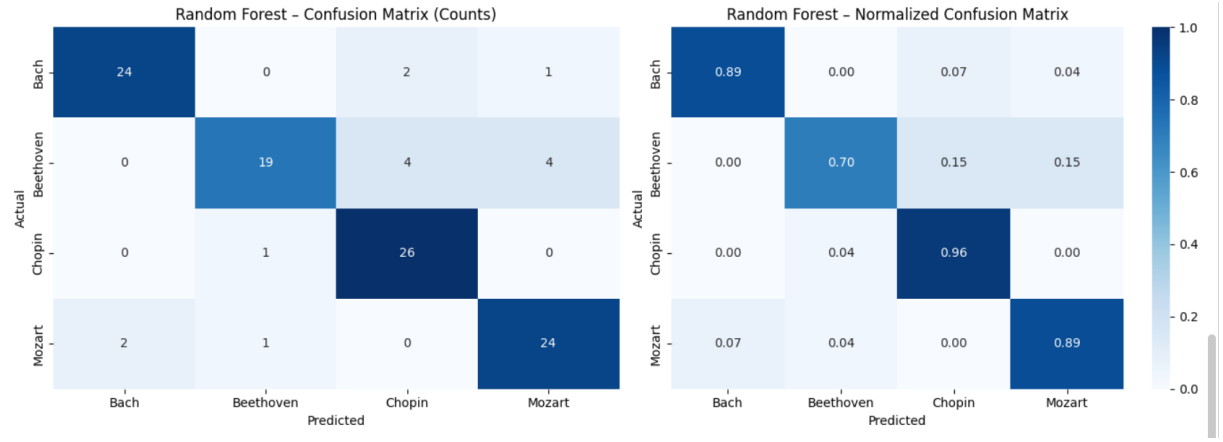
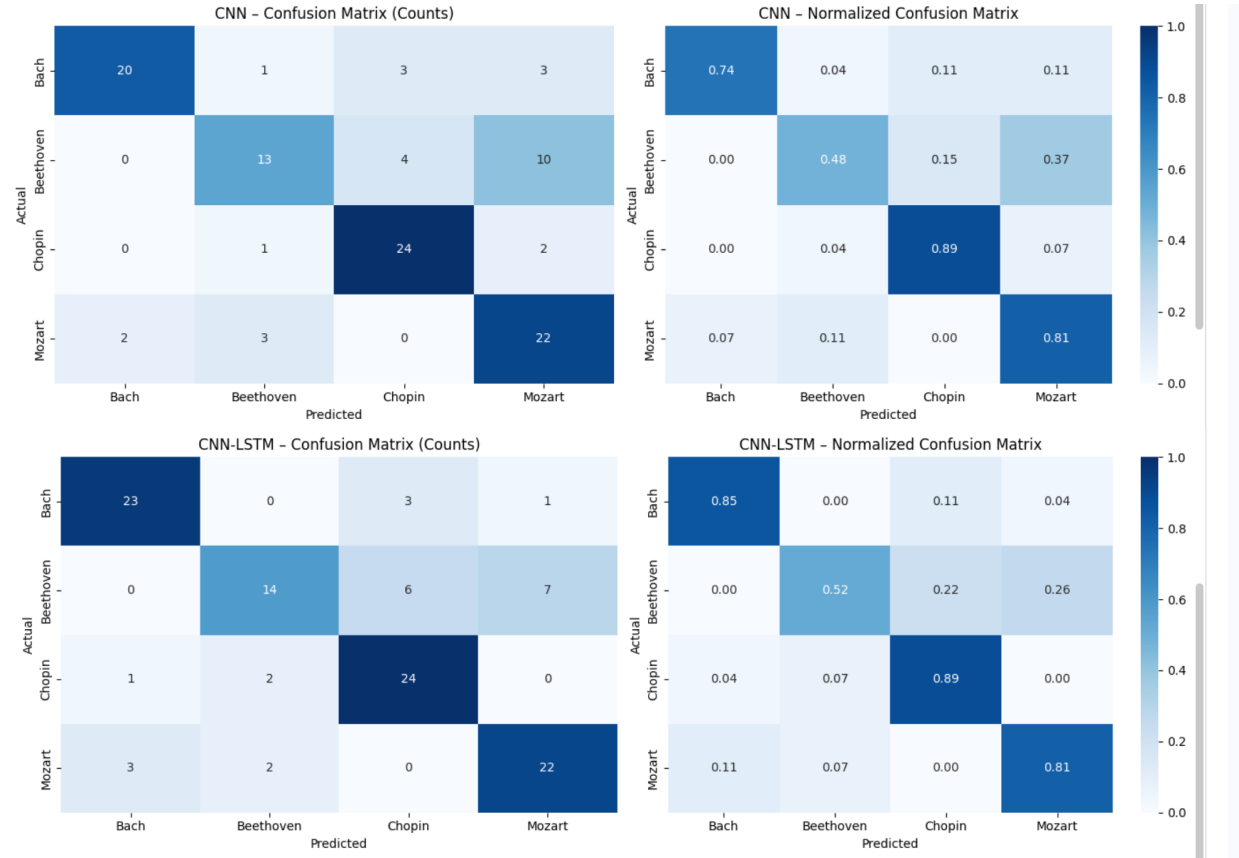


Figure 2

CNN and CNN-LSTM Confusion Matrices for the 10-Second, 20-FPS Runs



The deep-model confusion matrices show that Beethoven was the most difficult composer to separate. Beethoven was most frequently predicted as Mozart in both models and was also confused with Chopin, especially by the CNN-LSTM. These errors suggest that the selected piano-roll representation and the available sample size did not always capture enough discriminative structure for the Beethoven class.

Segment-Length Optimization

We next compared 10-second and 15-second piano-roll excerpts while keeping the representation at 20 FPS. Longer context improved the CNN but did not improve the CNN-LSTM. The 15-second CNN became the strongest pure deep-learning configuration tested, although it still remained below the Random Forest and support vector machine.

Table 2

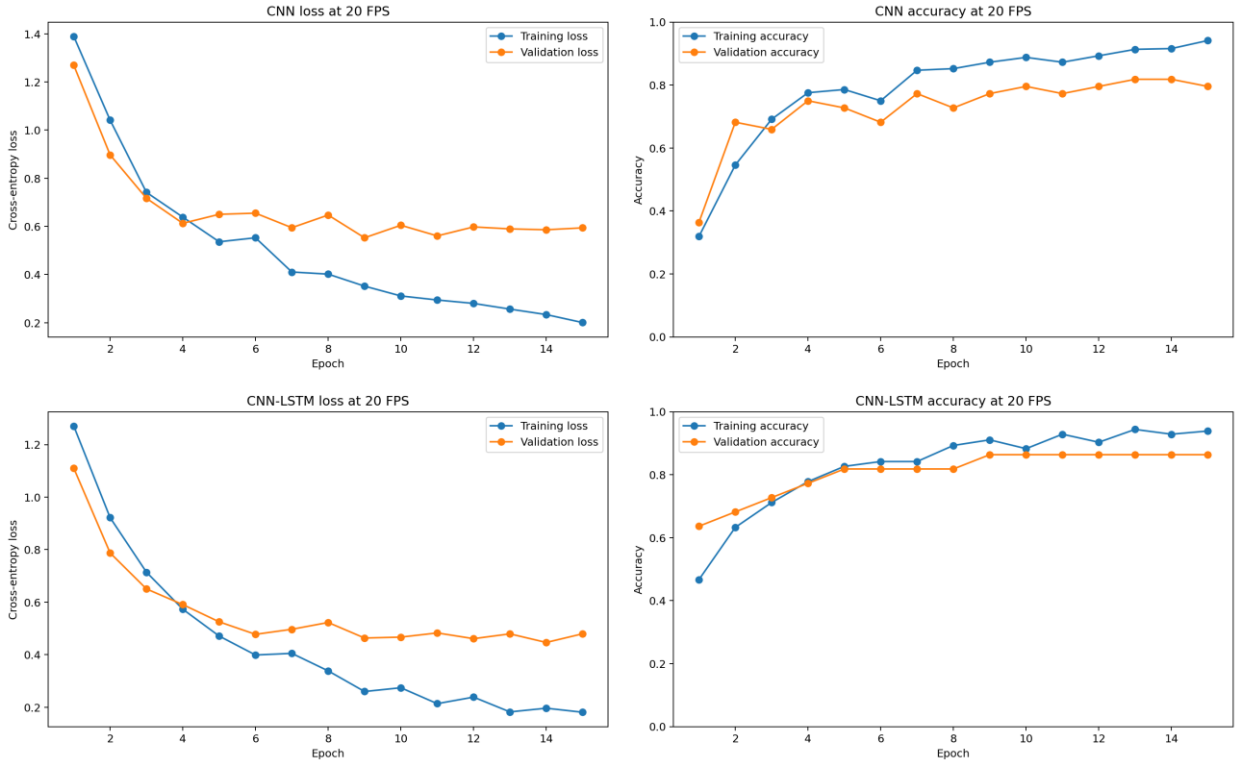
Deep-Model Segment-Length Comparison

Model	Segment Length	Accuracy	Macro-F1
CNN	10 s	0.7315	0.7273
CNN	15 s	0.7778	0.7776
CNN-LSTM	10 s	0.7685	0.7615
CNN-LSTM	15 s	0.7500	0.7485

Increasing the CNN input from 10 seconds to 15 seconds improved macro-F1 from 0.7273 to 0.7776, a gain of about five percentage points. In contrast, the CNN-LSTM decreased from 0.7615 to 0.7485 macro-F1. The result suggests that additional context helped the convolutional model, while the recurrent model did not convert the longer sequence into better test performance.

Figure 3

Training and Validation Curves for the 10-Second, 20-FPS CNN and CNN-LSTM



Follow-Up Experiments

Several follow-up experiments tested whether the deep representations or additional data could close the gap. Stronger pitch, time, and velocity augmentation reduced the 15-second CNN macro-F1 from 0.7776 to 0.7579. A hybrid Random Forest trained on CNN embeddings concatenated with engineered features reached 0.7866 macro-F1, and the hybrid logistic regression reached approximately 0.7850. Both hybrid models improved on the pure deep models but remained below the original Random Forest and support vector machine.

Table 3

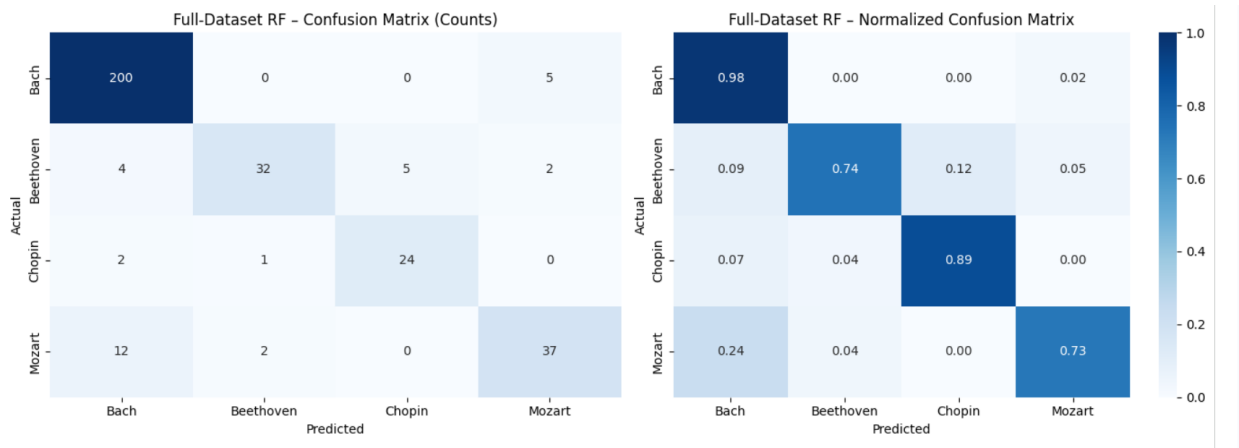
Follow-Up Experiment Results

Experiment	Dataset	Accuracy	Macro-F1
CNN + stronger augmentation (15 s)	Balanced	0.7593	0.7579
Hybrid Random Forest	Balanced	0.7870	0.7866
Hybrid Logistic Regression	Balanced	0.7870	0.7850
Random Forest (full set)	Imbalanced	0.8988	0.8506

The full-dataset Random Forest used all 1,628 valid MIDI files with a stratified split of 1,171 training, 131 validation, and 326 testing files. This experiment used 400 trees and balanced class weights. Overall accuracy increased to 0.8988, but macro-F1 was 0.8506, slightly below the balanced Random Forest score of 0.8590. Bach achieved the strongest class-level F1 (0.9456), while Mozart remained the weakest (0.7789). The higher overall accuracy therefore reflected the influence of the much larger Bach class rather than a uniform improvement across composers.

Figure 4

Full-Dataset Random Forest Confusion Matrices



Discussion

The Random Forest result stands out across the experiments. On the shared balanced test set, a compact set of 42 symbolic summary features outperformed the CNN and CNN-LSTM configurations. This does not show that deep learning is unsuitable for composer classification; rather, it suggests that the available balanced sample size and the chosen piano-roll representation were not sufficient for the neural networks to surpass the engineered-feature baselines.

The optimization experiments provide useful evidence about where additional complexity did and did not help. Longer context improved the CNN but reduced CNN-LSTM performance.

Stronger augmentation increased validation performance without improving held-out test performance. Hybrid CNN embeddings improved on the pure deep models but still did not exceed the strongest classical baselines. Finally, using the full imbalanced dataset raised accuracy but slightly lowered macro-F1 relative to the balanced Random Forest, demonstrating why macro-averaged metrics were important for this task.

Several limitations should be considered. The MIDI files vary in encoding quality and instrumentation, and one public collection was used for all experiments. The balanced training set included only 98 files per composer, which limits the amount of variation available to the deep models. The project also did not evaluate alternative representations such as Mel spectrograms or event-based symbolic sequences. Future work could test residual or squeeze-and-excitation CNNs, transformer-style sequence models, multi-crop test-time evaluation, larger balanced datasets, and independent external test collections.

Conclusion

On the four-composer task required for this course, the Random Forest trained on 42 engineered MIDI features produced the strongest balanced performance, reaching 0.8611 accuracy and 0.8590 macro-F1. The support vector machine followed at 0.8020 macro-F1. The best pure deep-learning result came from the 15-second CNN, which reached 0.7778 accuracy and 0.7776 macro-F1. The CNN-LSTM performed best with 10-second inputs, and neither stronger augmentation nor hybrid CNN-engineered features surpassed the strongest classical models.

The full-dataset Random Forest increased overall accuracy to 0.8988 but produced a slightly lower macro-F1 of 0.8506, reinforcing the importance of balanced evaluation. For this dataset size and representation, carefully engineered symbolic features remained a strong and

computationally efficient baseline. Future improvements should emphasize larger balanced collections, alternative music representations, broader architecture searches, and evaluation on an independent dataset.

References

Fedorak, B. (2019). *midi_classic_music* [Data set]. Kaggle. https://www.kaggle.com/datasets/blanderbuss/midi-classic-music

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org/

Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C.-Z. A., Dieleman, S., Elsen, E., Engel, J., & Eck, D. (2019). Enabling factorized piano music modeling and generation with the MAESTRO dataset. *International Conference on Learning Representations*. https://openreview.net/forum?id=r1lYRjC9F7

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278-2324. https://doi.org/10.1109/5.726791

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*, 8024-8035. https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in

Python. *Journal of Machine Learning Research, 12*, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html

Raffel, C. (2016). *Learning-based methods for comparing sequences, with applications to audio-to-MIDI alignment and to symbolic music similarity* (Doctoral dissertation, Columbia University). https://doi.org/10.7916/D8N58MHV

Raffel, C., & Ellis, D. P. W. (n.d.). *pretty_midi* [Computer software]. https://craffel.github.io/pretty-midi/

Appendix

Team Contributions

- **Nicholas Alforque:** Data collection, MIDI parsing, extraction of the 42 symbolic features, and initial dataset organization.
- **Thomas Geraci:** Shared train/validation/test preparation, 20-FPS piano-roll generation, CNN and CNN-LSTM architecture design, training loops, 10-second and 15-second experiments, augmentation, and model checkpointing.
- **Cameron Aljilani:** Test-set evaluation, confusion-matrix analysis, model comparison tables, hybrid-feature experiments, full-dataset trial, and final report compilation.

All team members collaborated on Google Colab, reviewed one another's work, and contributed to the final report.

AI-tool acknowledgement: Code structure, experimental iterations, debugging, and report editing were developed with assistance from a large language model. The team reviewed the code outputs, selected the final results, and is responsible for the final analysis and conclusions.

Project repository: https://github.com/Sublymonal/team-4-511